

PROJECT DOCUMENTATION REPORT

SCRIPT-CONTROLLED ACCESS CONTROL LIST (ACL) SECURITY PLATFORM

Full Stack Development with MERN Stack

TABLE OF CONTENTS

- 1. Project Overview & Team Structure
- 2. Introduction & Problem Statement
- 3. Project Objectives & Scope
- 4. Key Features & Background Study
- 5. System & Module Architecture
- 6. Database Architecture & Data Schema
- 7. Security Implementation: Authentication, RBAC & Script-Controlled ACL
- 8. Directory Structure & Setup Instructions
- 9. API Documentation
- 10. User Interface & Testing Strategy
- 11. Known Issues & Future Enhancements
- 12. Conclusion

1. PROJECT OVERVIEW & TEAM STRUCTURE

1.1 Executive Summary

The Script-Controlled Access Control List (ACL) Security Platform is an enterprise-grade security solution developed using the MERN Stack (MongoDB, Express.js, React.js, Node.js). Modern organizations handle massive volumes of sensitive records requiring granular, contextual access rules. Traditional permission systems—such as static Access Control Lists (ACL) or rigid Role-Based Access Control (RBAC)—often fall short when evaluating conditional or field-level parameters.

This platform bridges that gap by integrating Script-Controlled ACL Logic, enabling continuous context-driven access evaluation (department alignment, record sensitivity levels, ownership validation) before granting access.

1.2 Team Structure & Responsibilities

Name	Role	Core Responsibilities
Sumethaa P	Lead MERN Architect & Project Management	Overall system design, architectural governance, sprint

Name	Role	Core Responsibilities
		management, and integration lead.
Sri Ranjana V	Frontend Developer	User interface engineering, React.js dashboard creation, state management, and user flow optimization.
Bhuvana Lakshmi V	Backend Developer	REST API design using Express.js/Node.js, core controller logic, middleware development, and authentication pipelines.
Deva Dharshini B	Database Administrator	MongoDB schema modeling using Mongoose, database optimization, indexing, and data security standards.
Hema Sree K	QA & Security Test Engineer	Unit testing, API validation, penetration testing, unauthorized privilege checking, and security audits.

2. INTRODUCTION & PROBLEM STATEMENT

2.1 Introduction

Data protection is a paramount concern for enterprise entities. While authentication confirms a user's identity, authorization dictates what data that authenticated user can view, edit, or delete. Static security policies struggle to adapt to multi-departmental constraints and dynamic security classifications. The Script-Controlled ACL Security Platform addresses this challenge by evaluating dynamic conditions at the precise moment data is requested.

2.2 Problem Statement

Contemporary organizational security frameworks face severe operational bottlenecks:

- **Unauthorized Access to Sensitive Records:** Overly broad permission assignments expose confidential records.
- **Static Permission Management:** System administrators must manually reconfigure access rules for shifting roles or departmental re-allocations.
- **Insider Threats:** Legitimate accounts accessing data outside their operational domain or clearance boundaries.

- **Absence of Real-Time Monitoring:** Lack of centralized oversight into access requests and security policy violations.
- **Compliance Drag:** Difficulty maintaining adherence to strict data protection regulations due to fragmented access logs.

3. PROJECT OBJECTIVES & SCOPE

3.1 Primary Objectives

1. **Script-Controlled Access Execution:** Evaluate dynamic access conditions (department matching, clearance level, and ownership) programmatically.
2. **Field-Level Protection:** Restrict records dynamically based on context and sensitive attribute fields.
3. **Data Security & Integrity:** Enforce token-based security and robust data encryption at rest and in transit.
4. **Comprehensive Audit Tracking:** Record user actions, entry attempts, and policy denials to ensure total accountability.
5. **High Scalability:** Leverage MERN architecture to support smooth enterprise horizontal scaling.

4. KEY FEATURES & BACKGROUND STUDY

4.1 Core Platform Features

- **User Authentication:** Secure registration, login, and token session management via JSON Web Tokens (JWT) and Bcrypt hashing.
- **Role-Based Access Control (RBAC):** Base authorization layer classifying users into defined organizational tiers.
- **Script-Controlled ACL Engine:** Execution middleware enforcing dynamic policy evaluations.
- **Record Ownership Validation:** Built-in checks ensuring users can manage resources they created or are explicitly assigned to.
- **Centralized Audit Logging:** Complete administrative logs capturing all read, write, update, and denial events.
- **Security & Analytics Dashboard:** Real-time visibility into active users, access trends, and security incidents.

4.2 Background Study & Comparative Analysis

Model	Mechanism	Key Limitations	Script-Controlled ACL Advantage
Traditional ACL	Direct attachment of user IDs to resources	Hard to scale; tedious manual updates	Dynamic rule evaluation without static lists
RBAC	Permissions linked strictly to assigned roles	Lacks context (e.g., department, ownership)	Adds dynamic context layers to base roles

Model	Mechanism	Key Limitations	Script-Controlled ACL Advantage
ABAC	Attribute-based policy evaluations	Complex setup and overhead administration	Lightweight, scriptable logic in MERN

5. SYSTEM & MODULE ARCHITECTURE

5.1 High-Level Architecture (MERN Stack)

- **Frontend Layer (React.js):** Provides a single-page interface (SPA) for dashboard interaction, analytics, and record access requests.
- **Backend Layer (Node.js & Express.js):** Serves RESTful API endpoints, processes application business logic, and hosts the security engine.
- **Security & ACL Engine:** Middleware intercepting incoming HTTP requests to validate JWTs and run the Script-Controlled ACL logic.
- **Database Layer (MongoDB & Mongoose):** NoSQL persistence layer holding operational records, user data, and system logs.

5.2 Frontend Modules

- **Authentication Module:** Login, session persistence, and token clearance.
- **Role & User Management UI:** Admin controls for managing user tiers and system access.
- **Record Management System:** Contextual viewing, editing, and submission interface.
- **Analytics & Audit Log Dashboard:** Real-time summary charts and event tracking tables.

5.3 Backend Responsibilities

- **API Routing & Controllers:** Mapping endpoints to relevant data controllers.
- **Security Middleware:** Context generation, token parsing, and policy execution before reaching endpoints.
- **Data Layer Validation:** Schema enforcement using Mongoose models.

6. DATABASE ARCHITECTURE & DATA SCHEMA

6.1 MongoDB Collections Overview

The database is structured around four primary entities: **Users**, **Roles**, **Records**, and **AuditLogs**.

6.2 Key Data Schemas

User Entity

```
{
  "_id": "ObjectId",
```

```
"name": "String",

"email": "String (Unique)",

"password": "String (Hashed)",

"role": "String (Admin | Manager | Employee | Security Officer)",

"department": "String",

"clearanceLevel": "Number",

"createdAt": "Timestamp"

}
```

Record Entity

```
{

  "_id": "ObjectId",

  "title": "String",

  "content": "String",

  "department": "String",

  "sensitivity": "Number (1-5)",

  "ownerId": "ObjectId (Ref: User)",

  "createdAt": "Timestamp"

}
```

Audit Log Entity

```
{

  "_id": "ObjectId",

  "userId": "ObjectId (Ref: User)",

  "action": "String (READ | WRITE | DELETE | DENIED)",

  "resourceId": "ObjectId",

  "ipAddress": "String",

}
```

```
"timestamp": "Timestamp"
```

```
}
```

7. SECURITY IMPLEMENTATION: AUTHENTICATION, RBAC & SCRIPT-CONTROLLED ACL

7.1 Authentication & RBAC Tiering

- **Authentication:** Password validation via `bcryptjs` paired with signed, time-bound JSON Web Tokens (JWT) passed via standard Bearer headers.
- **Role Hierarchy:**
 - **Admin:** Unrestricted access across system metrics and users.
 - **Security Officer:** Access to security logs, policy monitors, and auditing modules.
 - **Manager:** Elevated operational access within their designated department.
 - **Employee:** Basic operational access confined to assigned task records.

7.2 Script-Controlled ACL Engine Flow

Access is granted if:

```
$$\text{Access Granted} \iff (\text{Department}\{\text{User}\} = \text{Department}\{\text{Record}\} \land \text{Clearance}\{\text{User}\} \geq \text{Sensitivity}\{\text{Record}\}) \lor (\text{ID}\{\text{User}\} = \text{OwnerID}\{\text{Record}\})$$
```

Otherwise, access is denied and an audit log entry is immediately generated.

8. DIRECTORY STRUCTURE & SETUP INSTRUCTIONS

8.1 Repository Folder Layout

acl-security-platform/

```
├── client/           # Frontend - React.js
|
|   ├── src/
|   |   ├── components/  # Reusable UI components
|   |   ├── pages/       # View pages (Login, Dashboard, Records)
|   |   ├── services/    # Axios API connection services
|   |   └── App.js
|   └── package.json
```

```
└─ server/          # Backend - Node.js & Express
    └─ config/       # Database connection scripts
    └─ controllers/  # Request handling business logic
    └─ middleware/   # JWT & Script-Controlled ACL logic
    └─ models/       # Mongoose MongoDB schemas
    └─ routes/       # API route definitions
└─ server.js
```

8.2 Prerequisites & Setup

- **Prerequisites:** Node.js (v18+), MongoDB (Local or Atlas), VS Code, Git.
- **Installation:**
 - Run `npm install` inside the `server/` directory.
 - Run `npm install` inside the `client/` directory.

8.3 Running the Application

- **Backend:** Navigate to `server/` and run `npm start`.
- **Frontend:** Navigate to `client/` and run `npm start`.

9. API DOCUMENTATION

Method	Endpoint	Access Level	Description
POST ▾	/api/auth/login ▾	Public	Authenticates credentials and issues a JWT token.
GET ▾	/api/users ▾	Admin, Security Officer	Fetches user management lists.
POST ▾	/api/users ▾	Admin	Registers a new internal account.
GET ▾	/api/records ▾	Authenticated	Retrieves records filtered dynamically through the Scripted ACL.
POST ▾	/api/records ▾	Admin, Manager	Creates a record with sensitivity classifications.
DELETE ▾	/api/records/:id ▾	Owner, Admin	Removes a record after checking ownership/ACL rules.
GET ▾	/api/audit-logs ▾	Security Officer, Admin	Retrieves system security logs.

10. USER INTERFACE & TESTING STRATEGY

10.1 UI Dashboard Architecture

- **Secure Login Portal:** Password masked inputs with token handling.
- **Management Console:** Modular views displaying user tables, record management matrices, and department assignments.
- **Security Audit Screen:** Real-time visual logs displaying approved vs. rejected access attempts.

10.2 Testing Strategy

- **Unit Testing:** Individual controller function validation.
- **Integration Testing:** End-to-end testing of data flows between frontend, backend controllers, and MongoDB.
- **Security & Penetration Testing:** Testing unauthorized privilege escalation and cross-department access without ownership.

11. KNOWN ISSUES & FUTURE ENHANCEMENTS

11.1 Current Limitations

- Basic real-time risk score assignment.
- Lack of automated behavioral detection for repeated access failures.
- Data visualizations are static rather than predictive.

11.2 Future Roadmap

- **AI Risk Scoring Engine:** Incorporate machine learning models to dynamically compute threat levels per session.
- **Behavioral Analytics:** Detect abnormal user activity patterns.
- **Zero-Trust Architecture:** Continuous token re-verification on high-sensitivity data actions.
- **Predictive Threat Prevention:** Automated account lockdowns upon detecting suspicious brute-force or bulk read patterns.

12. CONCLUSION

The Script-Controlled ACL Security Platform successfully demonstrates how modern web architecture can enforce granular, dynamic authorization rules in enterprise settings. By combining the agility of the MERN Stack with custom script-controlled security logic, the platform prevents unauthorized access, reduces insider threats, and provides complete auditability across organizational data.